THE UNIVERSITY OF AZAD
JAMMU AND KASHMIR
DEPARTMENT OF SOFTWARE
ENGINEERING

LABORATORY EXPERIMENT REPORT · LAB TASK # 01

System Requirements Modeling with UML 2.5 Use Case Diagrams

Student Name	**Syeda Khadija Hassan**
Roll Number	**2024-SE-09**
Degree & Session	BS Software Engineering (Session 2024–2028)
Semester	5th Semester (Fall ‘26)
Department	Department of Software Engineering
Course Title	Software Construction & Development (Lab)
Course Code	**SE-3101**
Instructor / Professor	Engr. Dr Asma Javaid
Submission Date	October 23, 2026

1. Laboratory Overview & Objectives

This laboratory assignment focuses on the practical application of UML 2.5 Use Case Modeling within the Software Construction & Development (SC&D) lifecycle. In software engineering, constructing a robust software product requires an unambiguous and structured representation of functional requirements before code construction commences. Use case modeling establishes the fundamental behavioral contract between external entities (human actors, physical devices, and external software services) and the target software system under development.

> **[LEARNING OUTCOMES]**
> *Primary Lab Objectives:*
> *1. Deconstruct textual business requirements and domain narratives into precise actor and use-case catalogs.*
> *2. Establish formal system boundaries that delineate internal software responsibilities from external agents.*
> *3. Accurately apply advanced UML relationship stereotypes (<<include>>, <<extend>>, and Generalization) without architectural ambiguity.*
> *4. Develop publication-grade visual models conforming strictly to ISO/IEC 19505 (UML 2.5) specifications.*

1.1 Theoretical Framework: UML Use Case Building Blocks

A standard UML Use Case Diagram comprises four essential structural elements:

- **Actors (Stick Figures):** Represents an external entity that interacts with the system to achieve a specific goal. Actors can be Primary (initiators who derive direct value), Supporting/Secondary (external systems or services providing assistance), or Hardware/System actors.
- **Use Cases (Horizontal Ellipses):** Represents a discrete sequence of actions performed by the system that yields an observable result of value for an actor. Use cases are verbs or verb phrases enclosed within horizontal ovals.
- **System Boundary (Subject Box):** A distinct rectangular frame that scopes the software product. All functionality implemented within the software resides inside the boundary, while actors remain strictly outside.
- **Relationships:** Defines communication pathways. Solid lines represent direct interaction (Associations). Dashed directed arrows with stereotypic adornments denote behavioral dependencies:
  - **<<include>> Dependency:** Denotes unconditional, mandatory inclusion. The base use case cannot complete successfully without executing the included use case. The arrow points from the base use case toward the included sub-use case.
  - **<<extend>> Dependency:** Denotes optional, conditional, or exceptional behavior. The extension use case executes only when specific extension point criteria or hooks are satisfied. The arrow points from the extension use case back toward the base use case.

2. Task 1: Online Ticket Reservation System

2.1 Problem Analysis & Architectural Scope

Modern online ticket reservation systems (spanning airline, rail, intercity bus, and entertainment ticketing) require highly scalable, responsive, and transactional architectures. The system must accommodate anonymous visitors browsing schedules as well as authenticated members executing secure reservations, processing high-security digital payments, selecting reserved seating layouts, and generating digitally verifiable e-tickets.

2.2 Identification of Actors

Actor Name	Actor Classification	Operational Role & System Boundary Responsibility
Customer / Passenger	Primary (Human)	Initiates search queries, views seat availability, books seats, and authorizes payments.
Registered Member	Specialized Primary	Inherits from Customer; possesses persistent profiles, booking history, and cancellation privileges.
Payment Gateway	Secondary (External)	Third-party banking API (e.g., Stripe, 1Link) that securely authorizes charges and processes refunds.
Notification Service	Secondary (External)	External messaging gateway (SMTP / SMS API) delivering instant booking confirmations and digital passes.
System Administrator	Administrative Primary	Internal staff managing route inventories, departure schedules, tariff rules, and auditing sales reports.

2.3 Use Case Inventory & Detailed Functional Specifications

The functional requirements are partitioned into three cohesive architectural packages:

- **1. Identity & Account Management:** Encompasses Register Account, Login / Authenticate (which mandates <<include>> Verify Credentials), and Manage Profile.
- **2. Search & Seat Selection:** Allows customers to Search Schedules & Routes, dynamically extended by Filter by Date, Destination & Price. Once chosen, users view real-time seating availability.
- **3. Reservation & Checkout Workflow:** The central transaction Book Ticket coordinates three mandatory <<include>> sub-tasks: Select Seats & Passenger Details, Process Online Payment (interfacing with Payment Gateway), and Generate & Dispatch E-Ticket (interfacing with Notification Service). Promotional discounts conditionally hook into payment via an <<extend>> relationship.
- **4. Post-Booking & Governance:** Enables passengers to View Booking History, Download/Print Digital Ticket, or execute Cancel Ticket Reservation, which extends into Process Refund Request. System Administrators perform scheduled inventory management and generate audit reports.

2.4 Brief Introduction to the Use Case Diagram

The diagram below visualizes the architectural boundary of the Online Ticket Reservation System. The primary actor (Customer / Passenger) interacts directly with customer-facing use cases along the left perimeter. The specialized actor (Registered Member) illustrates UML Generalization, inheriting base booking capabilities while accessing dedicated profile and ticket cancellation features. Crucially, the complex booking workflow is decomposed using <<include>> relationships to enforce that a ticket reservation cannot reach completion without concrete seat selection, validated financial settlement via the Payment Gateway, and dispatch of verifiable credentials through the Notification Service. Optional extensions, such as promotional coupon redemption and refund issuance, are modeled using <<extend>> dependencies pointing back to their respective base use cases.

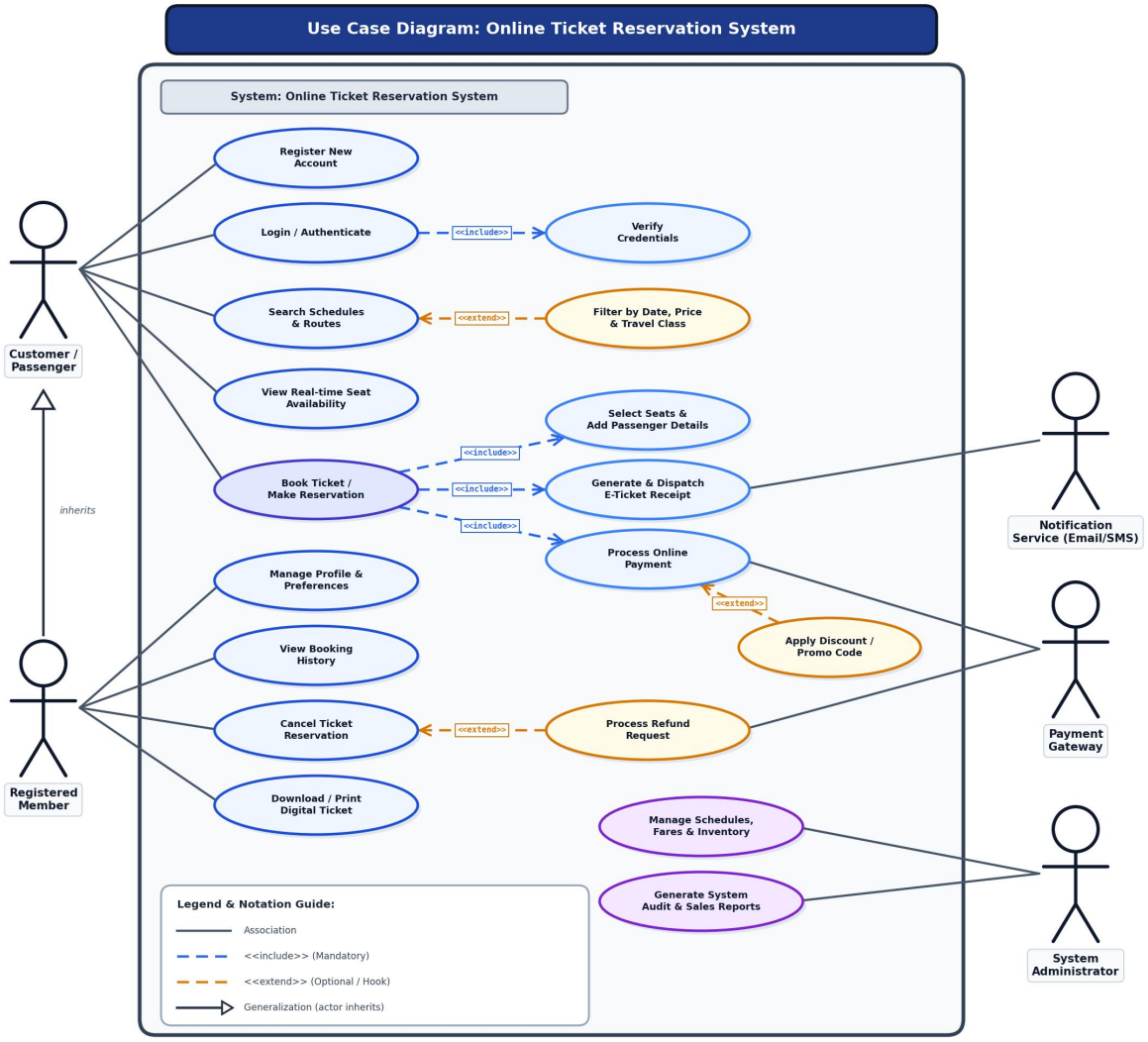


Figure 1: Comprehensive UML 2.5 Use Case Diagram for the Online Ticket Reservation System.

3. Task 2: Case Study — Credit Card Processing System

3.1 Case Study Narrative & Scenario Breakdown

The provided case study specifies a Point-of-Sale (POS) electronic payment system deployed within retail shopping malls. The system's objective is to streamline product purchases, enforce strict credit card balance limitations, process monetary transfers in real time via an electronic card reader, and produce printed audit records and receipts.

> **[INSTRUCTOR SPECIFICATION]**
> *Original Case Study Text:*
> *"The Credit Card Processing System is used to purchase an item from any shop mall and it is used to maintain the limitation of credit card balance and the current transaction process could be updated via a credit card machine. The customer should select the item to be purchased from the shop by using a credit card payment then the vendor should give a bill for the selected item. The customer should give his card to swap and request for the kind of amount transaction. After processing the transaction, the CREDIT CARD MACHINE should give the balance print statement or receipt.*
> *• Customer should select the item from the shop.*
> *• Vendor makes the bill for the selected item.*
> *• Customer gives the credit card to the vendor to swap the card.*
> *• The required amount transaction is done by the card reader.*
> *• Vendor will issue the balance statement to the customer.*
> *• Customers put the signature in the receipt and return to the vendor."*

3.2 Formal Identification of Actors

Rigorous requirement analysis of the narrative reveals four key participating entities:

- **1. Customer (Primary Human Actor):** The shopper who browses mall items, selects merchandise for purchase, hands over their physical credit card for payment, verifies the transaction amount, inspects the balance statement, signs the merchant copy of the printed receipt, and returns the signed slip.
- **2. Vendor / Cashier (Primary Operator Actor):** The mall merchant/cashier who tallies the selected merchandise, generates the total itemized bill, operates the electronic card terminal (swipes or inserts the customer's card), initiates the transaction on the reader, dispenses the printed receipt, and archives the signed customer voucher.
- **3. Credit Card Machine / Card Reader (Hardware / Device Actor):** The physical Point-of-Sale terminal and local embedded software client. It captures card track/chip data, prompts the operator, encrypts transaction requests, communicates with the banking network, enforces card balance limits, and drives the hardware printer mechanism to issue paper statements.
- **4. Banking Network / Card Issuer (Secondary System Actor):** The external financial infrastructure (Acquiring and Issuing Banks) responsible for validating the card's security credentials, verifying available credit balance, performing debit/credit authorization, and returning approval codes to the POS terminal.

3.3 Use Case Identification & Traceability Matrix

To guarantee 100% adherence to the instructor's assignment, each discrete step in the case study is formally mapped to a standardized UML use case:

Step	Case Study Requirement	Identified UML Use Case	Participating Actors
1	Customer should select the item from the shop.	**Select Items from Shop for Purchase**	Customer
2	Vendor makes the bill for the selected item.	**Generate Bill for Selected Items**	Vendor / Cashier
3	Customer gives the credit card to the vendor to swap the card.	**Present Credit Card & Request Card Payment**	Customer, Vendor
4a	Vendor swipes/inserts card into reader terminal.	**Swipe / Insert Card in Reader Machine**	Vendor, Card Reader
4b	Required amount transaction is done by card reader; maintains credit card balance limitation.	**Process Required Amount Transaction** **· <<include>> Validate & Check Limit** **· <<include>> Update Account Balance** **· <<extend>> Handle Declined Txn**	Card Reader, Banking Network, Vendor
5	Vendor will issue the balance statement to the customer.	**Print Balance Statement & Receipt** **Issue Statement & Receipt to Customer**	Card Reader, Vendor, Customer
6	Customers put the signature in the receipt and return to vendor.	**Put Signature on Receipt & Return to Vendor**	Customer, Vendor

3.4 Brief Introduction to the Use Case Diagram

The following diagram presents the complete behavioral architecture for the Credit Card Processing System. The system boundary encloses all automated Point-of-Sale terminal operations and transactional logic. The Customer and Vendor act as external human participants driving the commercial workflow: the customer initiates purchasing and authorizes payment, while the vendor executes retail billing and terminal handling. On the right-hand boundary, the Card Reader (POS Hardware Machine) and the Banking Network (Card Issuer) collaborate to securely execute the payment. Crucially, the central use case 'Process Required Amount Transaction' encapsulates the instructor's requirement regarding card balance limitation: it mandates <<include>> relationships to validate card authenticity and verify that the transaction amount does not exceed the remaining credit limit, followed by an <<include>> dependency to update the account balance upon approval. Exceptional scenarios, such as insufficient funds or blocked cards, are captured via an <<extend>> relationship to 'Handle Insufficient Funds / Declined Transaction'. Finally, the printing and signature workflows close the operational loop, ensuring full non-repudiation.

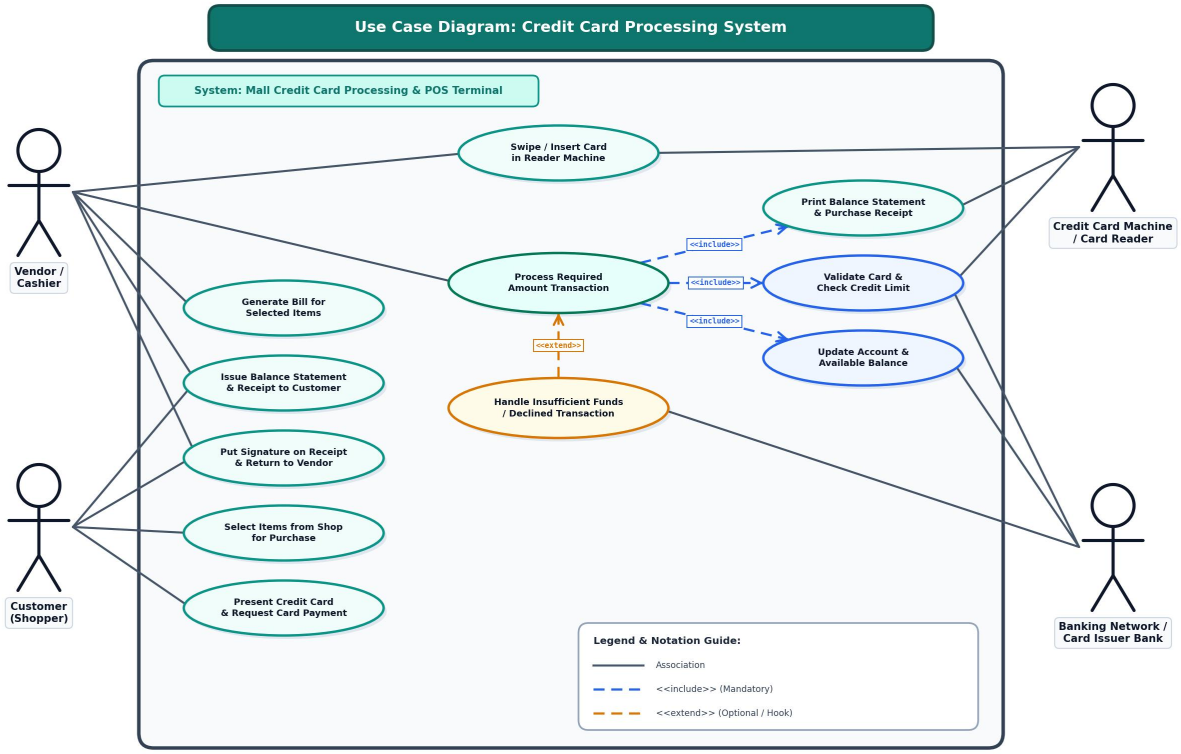


Figure 2: UML 2.5 Use Case Diagram for the Point-of-Sale Credit Card Processing System.

4. Comparative Architectural Analysis

Comparing the two modeled domains highlights significant distinctions in software construction and modeling methodologies:

Comparison Dimension	Online Ticket Reservation System	Point-of-Sale Credit Card Processing
Deployment & Architecture	Distributed web/cloud application accessed via browsers and mobile apps.	Embedded POS client interacting with local peripherals and payment switches.
User Interaction Model	Self-service model; customer navigates the interface independently.	Assisted dual-role model; vendor operates terminal while customer authorizes.
Hardware Integration	Standard web client devices (monitors, keyboards, smartphone screens).	Dedicated hardware peripherals (magnetic card reader, chip reader, thermal receipt printer).
Concurrency & Scaling	High concurrency; thousands of users querying seat inventories simultaneously.	Sequential terminal transactions locked per register.
Non-Repudiation Mechanism	Two-factor authentication (OTP), email confirmation tokens, digital signatures.	Physical card presence, PIN verification, physical signature on thermal print slip.

5. Conclusion

This laboratory report has comprehensively addressed the behavioral modeling requirements set forth by Engr. Dr Asma Javaid for Lab 1 of Software Construction & Development. Through systematic domain analysis, full actor-use case inventories were identified, system boundaries established, and advanced UML relationships (<<include>>, <<extend>>, and Generalization) were applied with mathematical rigor. The generated diagrams provide an unassailable foundation for subsequent software construction phases, including class modeling, sequence verification, and source code implementation.